

Housekeeping Module v1

Runbook & Technical Documentation

Module Overview

Purpose

The **Housekeeping module** manages room cleaning and turnover operations after guest checkout or manual operational triggers.

It ensures: - Rooms are not prematurely released to inventory - Cleaning work follows a strict lifecycle - Operational accountability for STAFF actions

This module is a **core operational pillar** between Reservations and Rooms.

Scope (v1)

Included: - Housekeeping task creation - Task lifecycle management - STAFF-driven execution - ADMIN oversight - Tenant isolation

Excluded (future versions): - Automatic task creation from reservations - SLA timers - Mobile push notifications - Maintenance workflows

Core Concepts

Tenant Isolation

- Tenant = Hotel
 - Every housekeeping task belongs to **exactly one tenant**
 - Cross-tenant visibility is forbidden
-

Housekeeping Task

A **Housekeeping Task** represents a unit of cleaning work for a room.

Tasks: - Are created manually (v1) - Are associated with exactly one room - Can be assigned and executed by STAFF

Entity Model

HousekeepingTask

Field	Type	Notes
id	UUID	Primary Key
tenant	FK → Tenant	Mandatory
room	FK → Room	Mandatory
assignedTo	FK → User	Nullable
status	enum	Lifecycle state
priority	enum	LOW / NORMAL / HIGH
notes	text	Optional
createdAt	timestamp	Auto
updatedAt	timestamp	Auto

Status Lifecycle (STRICT)

PENDING → IN_PROGRESS → COMPLETED



CANCELLED (ADMIN only)

Lifecycle Rules

- No skipping states
- No backward transitions
- COMPLETED is terminal

Role-Based Access Control (RBAC)

Action	ADMIN	STAFF
Create task	✓	✗
View tasks	✓	✓
Update status	✓	✓
Cancel task	✓	✗

Business Rules

Creation Rules

- Room must exist
- Tenant must exist
- Initial status = `PENDING`

Status Update Rules

- COMPLETED tasks cannot be modified
 - Invalid transitions are rejected
-

API Endpoints

Base Path

```
/housekeeping
```

Create Housekeeping Task

POST `/housekeeping?tenantCode=TENANT_CODE`

Auth: JWT (ADMIN)

Request Body

```
{
  "roomId": "uuid",
  "priority": "NORMAL",
  "notes": "Post checkout cleaning"
}
```

List Housekeeping Tasks

GET `/housekeeping?tenantCode=TENANT_CODE`

Auth: JWT (ADMIN, STAFF)

Update Task Status

PATCH `/housekeeping/:id/status`

Auth: JWT (ADMIN, STAFF)

Request Body

```
{  
  "status": "IN_PROGRESS"  
}
```

Error Handling

Scenario	HTTP
Missing / invalid JWT	401
Forbidden role	403
Tenant not found	404
Room not found	404
Task not found	404
Modify COMPLETED task	400

Test Cases (curl + jq)

Test Case 1 – Create Task (ADMIN)

```
curl -s -X POST http://localhost:3000/housekeeping?tenantCode=SUNU_HOTEL  
-H "Authorization: Bearer $ADMIN_TOKEN"  
-H "Content-Type: application/json"  
-d '{  
  "roomId": "ROOM_ID",  
  "priority": "NORMAL",  
  "notes": "Post checkout cleaning"  
}' | jq
```

Test Case 2 – List Tasks

```
curl -s http://localhost:3000/housekeeping?tenantCode=SUNU_HOTEL  
-H "Authorization: Bearer $ADMIN_TOKEN" | jq
```

Test Case 3 – Start Task (STAFF)

```
curl -s -X PATCH http://localhost:3000/housekeeping/TASK_ID/status
-H "Authorization: Bearer $STAFF_TOKEN"
-H "Content-Type: application/json"
-d '{ "status": "IN_PROGRESS" }' | jq
```

Test Case 4 – Complete Task (STAFF)

```
curl -s -X PATCH http://localhost:3000/housekeeping/TASK_ID/status
-H "Authorization: Bearer $STAFF_TOKEN"
-H "Content-Type: application/json"
-d '{ "status": "COMPLETED" }' | jq
```

Test Case 5 – Modify COMPLETED Task (FAIL)

```
curl -s -X PATCH http://localhost:3000/housekeeping/TASK_ID/status
-H "Authorization: Bearer $ADMIN_TOKEN"
-H "Content-Type: application/json"
-d '{ "status": "IN_PROGRESS" }' | jq
```

Expected:

```
{ "statusCode": 400 }
```

Module Status

Housekeeping Module v1 is LOCKED

- Code complete
- Runtime tested
- RBAC verified
- Documentation complete